bayes_oracle: a test case where the right answer is known

What this folder shows

With real traces, nobody knows the best possible success rate, so there is no way to tell how close a certificate is to the truth. This folder builds artificial traces where the best possible success rate is known exactly, and compares the BI of a set of models with that true value.

- `gaussian_oracle_suite.py`: each of the eight bits of the key byte leaks through its own noisy channel. The best possible attacker and its success rate are known in closed form. The paper reports a true rate of 0.989577 and a measured suite rate of 0.989503. Level A runs this script and checks both numbers.
- `bayes_oracle_synthetic.py`: a second artificial case with masking, used to compare networks of different widths.

Before you start

Only `requirements.txt`. No data. About twenty seconds.

Run

See the options:

```
python -m bayes_oracle.gaussian_oracle_suite --help
```

```
$ python -m bayes_oracle.gaussian_oracle_suite --help
usage: gaussian_oracle_suite.py [-h] [--output-dir OUTPUT_DIR]
                                [--n-seeds N_SEEDS] [--n-train N_TRAIN]
                                [--n-attack N_ATTACK] [--amplitude AMPLITUDE]
                                [--sigma SIGMA] [--delta DELTA]
                                [--ece-bins ECE_BINS]

options:
  -h, --help            show this help message and exit
  --output-dir OUTPUT_DIR
  --n-seeds N_SEEDS
  --n-train N_TRAIN
  --n-attack N_ATTACK
  --amplitude AMPLITUDE
  --sigma SIGMA
  --delta DELTA
  --ece-bins ECE_BINS
[exit code 0, 1 s]
```

Run with the paper's settings (they are also the defaults):

```
python -m bayes_oracle.gaussian_oracle_suite --amplitude 3.0 --sigma 1.0 \
    --n-train 102400 --n-attack 100000 --n-seeds 3 --output-dir results/bayes_gaussian_oracle_suite
```

```
$ python -m bayes_oracle.gaussian_oracle_suite --amplitude 3.0 --sigma 1.0 --n-train 102400 --n-attack 10000
    0 --n-seeds 3 --output-dir results/bayes_gaussian_oracle_suite
{
```

```
  "out_dir": "results/bayes_gaussian_oracle_suite",
  "rows": 6
}
[exit code 0, 16 s]
```

```
$ ls results/bayes_gaussian_oracle_suite
README.md
figures
gaussian_oracle_suite_rows.csv
gaussian_oracle_suite_summary.csv
[exit code 0, 0 s]
```

```
$ python -c "import pandas as pd; print(pd.read_csv('results/bayes_gaussian_oracle_suite/gaussian_oracle_sui
    te_summary.csv')[['model', 'p_bayes_mean', 'p_suite_mean', 'p_suite_plus_mean']].to_string(index=False))
    "
                   model  p_bayes_mean  p_suite_mean  p_suite_plus_mean
 bayes_oracle_known_means      0.989577      0.989577           0.991176
learned_gaussian_template      0.989577      0.989503           0.991108
[exit code 0, 1 s]
```

What to look for

The two numbers to compare are in the summary table. The true success rate is 0.989577 and the suite's measured rate is 0.989503. The gap is 0.00007, which shows that the suite is as good as the best possible attacker, up to the small margin that comes from testing on a finite number of traces. With the old defaults of the script (amplitude 2, 50,000 traces) the rate is about 0.83 and does not match the paper.